Performance Evaluation of Vector Indexing Techniques for AI Search Systems: D.3.1 Midterm Project Status Report

Devu Babu Sheeja
Data & Embeddings Lead
babushe@uwindsor.ca

Ashwin Senthur Pandian
Vector Indexing & Tuning Lead
senthur@uwindsor.ca

Rishi G. Patel
Query & Evaluation Lead
patel3zc@uwindsor.ca

Nikhil Goud Nathi
Experiments & Visualization Lead
nathin@uwindsor.ca

Master of Applied Computing, University of Windsor | COMP.8157: Advanced Database Topics (2026S) | Group 4 | July 3, 2026

Abstract

This report documents progress on a workload-consistent benchmarking framework for evaluating Flat Search, IVF, and HNSW vector indexes under identical experimental conditions using FAISS and deterministic Sentence-BERT embeddings over the ABC News Headlines corpus. As of this checkpoint, eleven of sixteen planned tasks are complete: the full data and embedding pipeline, the automated benchmarking and visualization system, and two of the three indexing methods (Flat and IVF), which have already been benchmarked on twenty thousand real headlines and across a scalability sweep from one thousand to one hundred thousand vectors. The remaining work, HNSW construction and the parameter-tuning experiments, is scoped and on schedule for the weeks ahead.

Index Terms*—Vector Databases, Semantic Search, Approximate Nearest Neighbour (ANN), HNSW, IVF, FAISS, Vector Indexing, Project Status*

I. Overall Project Health

Overall project health is assessed as **on track**. Preprocessing, embeddings, benchmarking automation, and visualization are complete, and the Flat and IVF indexes are built, validated, and already producing real measurements on the target dataset. HNSW construction and the parameter-tuning sweeps are the pieces still in progress, and both are scheduled comfortably within the remaining course timeline ahead of the final deliverables on July 26 and the in-class presentation on July 28.

The project builds a framework that compares Flat Search, IVF, and HNSW under the same corpus, embedding model, query workload, and hardware, measuring query latency, recall@K against an exact ground truth, memory footprint, index build time, and scalability behaviour, so that any observed difference between methods is attributable to the indexing structure itself rather than inconsistent test conditions.

II. Related Work

Malkov and Yashunin introduced HNSW, a graph-based approximate nearest-neighbour technique enabling high recall at low query latency through hierarchical graph traversal, though their evaluation focuses on algorithm-level performance rather than a unified database-benchmarking methodology [1]. Johnson et al. developed FAISS, a scalable similarity-search library supporting Flat, IVF, product quantization, and HNSW indexes with optimized CPU and GPU implementations, but FAISS itself does not prescribe a methodology for comparing indexing methods fairly [2]. Reimers and Gurevych introduced Sentence-BERT, the sentence-embedding model this project uses to map text into a semantically meaningful vector space for retrieval [3]. Commercial systems such as Pinecone, Milvus, and pgvector demonstrate the practical relevance of vector indexing, but their published evaluations are typically application-specific rather than designed for reproducible, workload-consistent academic comparison. This project addresses that gap by holding the dataset, embedding model, query workload, similarity metric, and execution environment fixed across all three indexing methods within every experiment.

III. System Architecture

The framework follows the five-layer architecture proposed at the full-proposal stage, implemented as independent, testable modules so that experimental variables stay isolated between layers. The *data ingestion and preprocessing layer* loads raw headlines, applies cleaning and normalization, and segments text into consistent units suitable for embedding. The *embedding generation layer* transforms each segment into a dense 384-dimensional vector using Sentence-BERT, with all randomness seeded so that embedding generation is deterministic and reproducible across repeated runs. The *vector indexing layer* builds Flat, IVF, and (in progress) HNSW indexes from the same embedding set under a common interface, so every index exposes an identical build/search contract to the layers above it regardless of its internal structure. The *query processing engine* converts held-out test queries into vectors with the same embedding model used at indexing time, then executes each query against every index under identical runtime conditions. The *evaluation and analytics layer* collects per-query latency, recall@K, memory footprint, and build time from each run, aggregates them across repeated experiments and dataset sizes, and renders the comparative and scalability visualizations reported in Section VI.

This modular separation is what allows the project to report progress honestly at a checkpoint such as this one: because each layer is independently testable, the team can state precisely which layers are complete (data ingestion, embedding, benchmarking automation, evaluation, and two of three indexing methods) and which remain in progress (the HNSW index and the parameter-tuning experiments) without the completed portions depending on unfinished work.

IV. Project Progress Breakdown

Table I summarises the sixteen-task breakdown across five phases: data collection and preprocessing, embedding generation, index construction, benchmarking and evaluation, and analysis and documentation. Tasks are sequenced by pipeline dependency; eleven are complete, demonstrating measurable progress since initiation.

#	Task	Owner	Status
1	Dataset selection & justification	Devu	Done
2	Data ingestion & preprocessing	Devu	Done
3	Sentence-BERT embedding generation	Devu	Done
4	Scaled corpora (1K-100K)	Devu	Done
5	Config-driven benchmark runner	Nikhil	Done
6	Flat index (ground truth)	Ashwin	Done
7	IVF index (k-means, nprobe)	Ashwin	Done
8	Query engine (timing, recall@K)	Rishi	Done
9	Benchmark orchestration	Rishi	Done
10	Initial Flat/IVF benchmarking	Rishi	Done
11	Visualization system	Nikhil	Done
12	HNSW index	Ashwin	WIP
13	Parameter-tuning sweeps	Ashwin	WIP
14	Full HNSW benchmarking	Rishi	Remaining
15	Final results write-up	Team	Remaining
16	Final demo & presentation	Team	Remaining

TABLE I. TASK BREAKDOWN STRUCTURE (WIP = IN PROGRESS)

All course milestones to date have been met on schedule: the pre-proposal (May 17), the full proposal and its recorded presentation (May 31 and June 2), and this midterm report. Table II summarises milestone status against the course timeline.

Milestone	Target	Status
D.1 Pre-proposal	May 17	Complete
D.2.1 Full proposal + presentation	May 31 / Jun 2	Complete
D.3.1 Midterm status report	Jun 28	Complete
HNSW + tuning complete	Mid-July	Upcoming
D.4/D.5 Final demo + report	Jul 26	Not started
D.6 In-class presentation	Jul 28	Not started

TABLE II. MILESTONE REVIEW

V. GitHub Repository and Version Control Evidence

Development is tracked in a shared GitHub repository, with tasks additionally organised on a GitHub Projects board split into four ownership areas. Repository: github.com/Devz0201/COMP8157-Vector-Indexing.

The repository currently holds only the modules marked *Done* in Table I: the data and preprocessing scripts, the embedding module, the Flat and IVF indexes, the query engine and benchmark runner, the visualization code, the experiment configurations, and the README. The HNSW index and the parameter-tuning script remain local pending testing and will be pushed once complete, so the repository accurately reflects verified progress rather than work in an untested state.

Each member uploads and commits their own completed work under their own account, matching the ownership in Table I, giving commit-history evidence of distributed, non-overlapping contribution. Devu Babu Sheeja (GitHub: Devz0201) is responsible for data_ingestion.py, prepare_headlines.py, and embeddings.py, having selected and justified the dataset and built the deterministic preprocessing and embedding pipeline. Ashwin Senthur Pandian (GitHub: ashwin-0707) is responsible for indexing.py, having implemented the Flat and IVF indexes with adaptive cluster scaling, with HNSW and tune_parameters.py in progress. Rishi Gaurangkumar Patel (GitHub: Rishihi29) is responsible for query_engine.py and benchmark.py, having implemented per-query timing, recall@K measurement, and orchestration across single and scalability runs. Nikhil Goud Nathi (GitHub: nikhilnathi2003) is responsible for run_benchmark.py and visualization.py, having implemented the config-driven CLI, the comparative and scalability charts, and the repository and project-board setup.

A. Evidence of Distributed Contribution

Figs. 1–4 show each member's file committed to the shared repository under their own GitHub account, confirming that the ownership described above is reflected in the actual commit history rather than asserted only in this report.

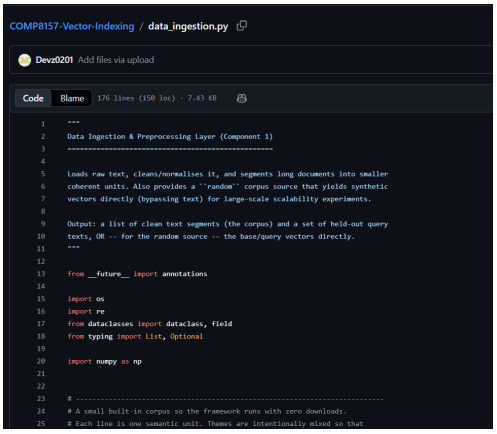

Fig. 1. data_ingestion.py committed by Devu Babu Sheeja (Devz0201).

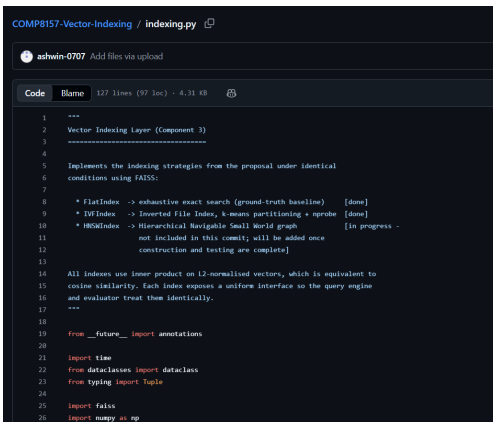

Fig. 2. indexing.py committed by Ashwin Senthur Pandian (ashwin-0707), showing the Flat/IVF implementation with HNSW noted as in progress.

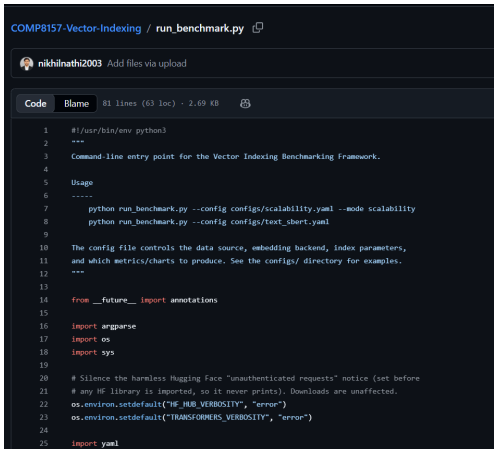

Fig. 3. run_benchmark.py committed by Nikhil Goud Nathi (nikhilnathi2003).

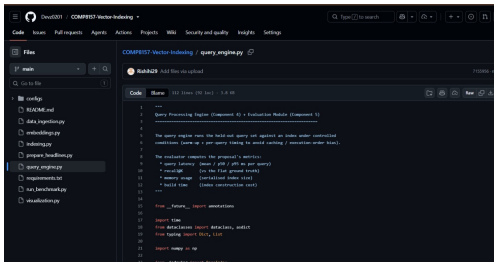

Fig. 4. query_engine.py committed by Rishi Gaurangkumar Patel (Rishihi29), viewed in the shared repository alongside the other completed modules.

VI. Demonstration of Completed Work

A. Metric Definitions

Query latency is reported as the wall-clock time to answer a single query, summarised per method as the mean, median (p50), and 95th-percentile (p95) across all held-out queries, after discarding a small warm-up set. Recall@K is defined as the fraction of a method's top-K returned items that also appear in Flat Search's exact top-K for the same query, averaged over all queries; a value of 1.000 indicates the approximate method returned exactly the same neighbours as exhaustive search. Index build time is the wall-clock time to construct the index structure from the embedded corpus, including any training step (such as IVF's k-means clustering). Memory footprint is measured from the serialized size of the built index. Together these four metrics, evaluated identically across Flat, IVF, and HNSW, form the basis for every comparison reported in this document.

B. Live Demonstration

A working prototype is available and demonstrable live. Executing *python run_benchmark.py* with a chosen configuration file loads the corpus, generates embeddings, builds the Flat and IVF indexes, executes two hundred held-out queries per method with warm-up and per-query timing, computes recall@10 against the exact Flat ground truth, and emits a results table with comparative charts.

C. Results

On twenty thousand real headlines embedded with Sentence-BERT, Flat Search answers a query in approximately 1.33 ms with recall of 1.000 (the exact baseline), while IVF answers in approximately 0.19 ms, roughly seven times faster, at the same recall and comparable memory (≈29.6 MB). Across the scalability sweep (1K–100K vectors), Flat latency grows linearly from ≈0.08 ms to ≈7.4 ms, the expected O(n) behaviour of exhaustive search, while IVF remains near 1.2 ms at 100K vectors with recall of 1.000 throughout.

A practical issue surfaced during scalability testing: at small corpus sizes, IVF's requested cluster count could exceed what k-means can reliably train on. This was resolved by adaptively scaling the cluster count to the dataset size, keeping IVF valid and warning-free across the full 1K–100K range.

The evaluation protocol is designed so results are directly comparable across methods. Every index is built from the same corpus and the same embeddings, and every method answers the same fixed set of held-out queries. Each query run is preceded by a small number of warm-up queries, discarded before timing begins, so that one-off costs such as lazy memory allocation do not bias the measured latency. Per-query latency is recorded individually rather than only as a batch average, which is what allows the framework to report the mean, median (p50), and 95th-percentile (p95) latency rather than a single aggregate number that could hide tail-latency behaviour. Recall@10 is computed by comparing each method's top-10 results against Flat Search's exact top-10 for the same query, so accuracy is always measured relative to a verifiable ground truth rather than an assumed one.

This consistency is what distinguishes the framework from simply re-running FAISS's own examples: because the corpus, embedding model, query set, similarity metric, and hardware are held fixed across all three methods within a run, any measured difference in latency, recall, memory, or build time is attributable to the indexing method itself, which is precisely the workload-consistent comparison identified as missing from prior tutorials and papers surveyed during the proposal stage.

Index construction cost and memory footprint are recorded alongside the query-time metrics, since a method that is fast to query but expensive to build or store is a different practical trade-off than one that is fast on every axis. On the real-headline benchmark, Flat requires negligible build time (under 0.02 s) since it performs no training, while IVF's k-means training adds a modest build cost (on the order of 0.2 s at 20K vectors) in exchange for its latency reduction; memory footprint for both is comparable (≈29–30 MB) since IVF stores the same raw vectors as Flat plus a small cluster-assignment overhead. These figures will be extended with HNSW's build time and memory once construction is complete, which is expected to show a larger build-time cost in exchange for its graph-based query speed, consistent with the trade-off documented in the HNSW literature [1].

VII. Risks and Action Plans

Category	Item / Action Plan
Known issue	IVF cluster count exceeding trainable minimum at small N; resolved via adaptive nlist scaling.

Known issue	Early recall inconsistency across repeated IVF runs; traced to unseeded corpus sampling and fixed by seeding all randomness.
Potential risk	Flat Search latency grows steeply at scale; mitigated by using Flat solely as the ground-truth baseline, not a proposed production method.
Potential risk	HNSW and tuning are the final implementation items with ~4 weeks remaining; prioritized early in the next period to preserve time for analysis and writeup.
Change request	None; the approach approved in the Full Proposal is being followed as planned.

TABLE III. RISK REGISTER AND MITIGATION

VIII. Environment and Tools

The framework is implemented in Python 3.13 using FAISS for index structures, sentence-transformers (all-MiniLM-L6-v2) for embeddings, and NumPy, Pandas, and Matplotlib for data handling and visualization. Experiments are defined declaratively in YAML so runs are fully reproducible from the command line, and all randomness is seeded, which is essential for the framework's core purpose of producing a fair, repeatable comparison across indexing methods.

Reproducibility is treated as a first-class requirement rather than an afterthought, since the project's central claim, that observed differences between indexing methods reflect the methods themselves, only holds if every run is genuinely repeatable. Each configuration file fully specifies the dataset, embedding backend, index parameters, and evaluation settings for a run, so no manual code editing is needed between experiments and the exact configuration used for any reported result can be checked into version control alongside the code that produced it. Development is carried out locally by each member and synchronized through the shared GitHub repository referenced in Section V, with the GitHub Projects board tracking which of the sixteen Table I tasks each member is currently working on.

Future Work

Beyond completing HNSW and the parameter-tuning sweeps, the framework is designed to extend to additional indexing techniques such as Product Quantization (PQ), ScaNN, and DiskANN without modifying the core benchmarking pipeline, since every index need only implement the same build and search interface used by Flat, IVF, and HNSW. Later work could also broaden the corpus beyond news headlines to longer-form documents or multimodal embeddings combining text with images, and could evaluate indexing behaviour under distributed or cloud-native deployment rather than a single-machine setup, which would provide additional insight relevant to production-scale AI-native database systems.

IX. Summary

The team is on schedule and has, at this checkpoint, more implemented and validated than strictly required: two indexing methods are complete, benchmarked on real data, and verified against an exact ground truth, with the full pipeline reproducible from a single command. The remaining work, HNSW, parameter tuning, full results analysis, and the final report, is well scoped, with no current blockers to completion ahead of the July 26 and July 28 deadlines.

Acknowledgment

The team thanks Dr. Andreas S. Maniatis for guidance on the project scope and evaluation methodology during the proposal and peer-review stages of COMP.8157, and acknowledges the ABC News organization and the maintainers of the “A Million News Headlines” dataset for making the corpus used in this study publicly available under a CC0 licence.

References

[1] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.

[2] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[3] R. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019.

[4] “A Million News Headlines,” ABC News (Kaggle / Harvard Dataverse), CC0 1.0, dataset used as the experimental corpus for this project.